PARIS CITE UNIVERSITY

Université Paris Cité

RESEARCH PROJECT

Self-Supervised Test-Time Training for Image Classification

MAY 2026

Authors:

Rayane KHATIM
rayane.khatim@etu.u-paris.fr
22500285

Mehdi AGHAEI
mehdi.aghaei@etu.u-paris.fr
22500371

Supervisors:

Prof. Ayoub Karine : ayoub.karine@u-paris.fr
Prof. Camille Kurtz : camille.kurtz@u-paris.fr
Prof. Laurent Wendling : Laurent.Wendling@u-paris.fr

Contents

1 Introduction and Context

Image classification models usually work well when the test images are close to the images used during training. This assumption is weak in practice. A camera image can contain sensor noise, blur, compression artifacts, fog, snow, low contrast, or a strong change in brightness. These changes do not necessarily change the object class, but they change the input distribution seen by the model. A network trained only on clean images can then make unstable predictions, even when the image is still understandable for a human observer.

This project studies that problem in a controlled computer vision setting. CIFAR-10 is used as the clean source domain [6]. The target domain is based on corrupted CIFAR-10 images, following the CIFAR-10-C evaluation idea [4]. CIFAR-10-C is useful because it separates common corruptions into named corruption types and severity levels. It is not an adversarial benchmark. The corruptions are ordinary image degradations: noise, blur, weather effects, digital compression, contrast changes, and geometric distortions.

The project does not try to design a new backbone architecture. The backbone is ResNet18 [7], and the representation is trained with SimCLR, a self-supervised contrastive method [1]. The main question is narrower: after the model has been trained on source data, can it adapt itself at test time using only an unlabeled corrupted batch? If this adaptation improves accuracy, the model becomes less dependent on perfectly clean test data. If it decreases accuracy, the adaptation objective is not aligned well enough with classification.

The adaptation strategy implemented in the project is ActMAD-style activation matching [5]. The model first records activation statistics on clean source data. At test time, it receives a corrupted batch and updates a limited subset of parameters so that the target activations become closer to the source activations. The current implementation adapts normalization parameters and keeps the rest of the model frozen. The classification head is then applied after this short adaptation.

The self-supervised configuration schedules SimCLR training for 200 epochs. In the training log, the linear evaluation accuracy reached 84.23%. On CIFAR-10-C severity 5, the final all-corruption evaluation improved the mean accuracy from 54.82% without adaptation to 70.22% after ActMAD TTT. This corresponds to an average gain of 15.40 percentage points. TTT helped on 14 of the 15 corruptions and hurt only on brightness.

2 Scientific Background and Related Work

2.1 Supervised image classification and distribution shift

In supervised image classification, the model learns a function

$$f_\theta : \mathcal{X} \to \{1, \dots, K\}$$

from images to class labels. For CIFAR-10, $K = 10$. The usual training objective assumes that the training and test examples are sampled from similar distributions. Under that condition, minimizing the training loss can lead to good test accuracy if the model generalizes correctly.

The problem changes when the test distribution is shifted. A corrupted image may still contain the same semantic object, but the low-level statistics can be different. Noise modifies pixel values. Blur removes high-frequency details. Compression creates blocks and artifacts. Fog and brightness changes affect contrast and color distributions. Since convolutional neural networks learn a mixture of low-level and high-level features, these changes can propagate through the network and modify the internal activations.

CIFAR-10-C was introduced as part of the common corruptions benchmark proposed by Hendrycks and Dietterich [4]. The benchmark is useful because it avoids vague robustness claims. A method can be tested on fixed corruptions and severity levels. If the full benchmark is used, accuracy or error can be reported per corruption and then averaged.

2.2 Self-supervised representation learning

Self-supervised learning uses labels created from the data itself. The goal is not to predict the final class label directly, but to learn a representation that contains useful structure. After pretraining, a smaller supervised head can be trained on top of the frozen representation. This is often called linear evaluation when the head is a linear classifier.

SimCLR is a contrastive self-supervised method [1]. For each image, two different augmented views are generated. The model should map these two views close to each other in representation space. Views from different images should be separated. This gives a training signal without class labels. For this project, SimCLR is a good fit because CIFAR-10 can provide many augmented image pairs, and the learned backbone can later be reused for classification and test-time adaptation.

The implementation uses a ResNet18 backbone and a projection head. The backbone produces a feature vector. The projection head maps this feature vector into the contrastive space used for the SimCLR loss. The classifier is not trained during contrastive pretraining. It is trained later during linear evaluation on frozen features.

2.3 Contrastive learning and NT-Xent loss

Let x_i be an image and let t_1 and t_2 be two stochastic augmentations. SimCLR builds two views:

$$\tilde{x}_{i,1} = t_1(x_i), \qquad \tilde{x}_{i,2} = t_2(x_i).$$

The encoder and projection head map them to normalized vectors $z_{i,1}$ and $z_{i,2}$. These two vectors are a positive pair. The other views in the same batch act as negative examples.

The project uses the NT-Xent loss. For a positive pair (i, j), a simplified form is:

$$\ell_{i,j} = -\log \frac{\exp(\mathrm{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\mathrm{sim}(z_i, z_k)/\tau)},$$

where sim is a similarity score and τ is the temperature. In the code, the projected vectors are normalized and the similarity matrix is computed by a dot product. The diagonal is masked because each view should not be compared with itself. The target label for each view is the index of its paired augmented view.

This loss is not a classification loss. It does not know the CIFAR-10 class names. It only asks the network to learn invariances induced by the augmentation pipeline. If the augmentations are well chosen, the representation captures useful image structure. If the augmentations are too weak or too strong, the representation can become less useful for classification.

2.4 Linear evaluation

Linear evaluation is a practical check of representation quality. After SimCLR pretraining, the backbone is frozen. A linear classifier is trained on top of the features using class labels. A high linear evaluation accuracy means that the frozen representation already separates classes well enough for a simple head. A low linear evaluation accuracy means that the representation is weak or not aligned with the classification task.

This matters for TTT. Test-time adaptation cannot fully repair a poor representation. If the backbone does not extract useful features before adaptation, then matching activation statistics may not be enough. The recorded run reached 84.23% linear evaluation accuracy. This is useful evidence that the encoder learned class-relevant features, but it is still below what a strong fully supervised CIFAR-10 model would usually obtain.

2.5 Test-Time Training and test-time adaptation

Test-Time Training adapts a model during inference using an auxiliary objective that does not need labels [2]. A model trained on a source domain receives target-domain inputs at test time. Before predicting labels, it performs a few gradient steps on a self-supervised or unsupervised loss. The goal is to move the model toward the target distribution without using target labels.

The difficulty is that the auxiliary loss is not the classification loss. It may improve the internal representation, but it may also move the model in a harmful direction. TTT++ studies this instability and shows that self-supervised test-time training can fail under some shifts [3]. A positive result on one evaluated setting is useful, but it does not prove that the method is safe for all corruptions.

Test-time adaptation is attractive because it does not require labeled target data. In many applications, labeled target data is not available when the distribution changes. The model only sees new unlabeled images. This matches the project setting: target batches from corrupted CIFAR-10 images are available, but their labels should not be used for adaptation.

2.6 ActMAD and activation matching

ActMAD proposes to adapt a model by matching activation distributions between source data and test-time data [5]. The intuition is that a distribution shift changes internal activations. If the model can adjust itself so that target activations become closer to source activations, its predictions may become more stable.

The project implements a simplified ActMAD-style objective. During a source-statistics phase, hooks record the activations of normalization layers on clean data. For each layer, the code computes a mean and a standard deviation. During adaptation, the target batch is passed through the model,

target activation statistics are computed, and the loss penalizes the difference between target and source statistics. In simplified notation, for layers $l \in \mathcal{L}$:

$$\mathcal{L}_{\text{ActMAD}} = \frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} \left(\|\mu_l^t - \mu_l^s\|_2^2 + \|\sigma_l^t - \sigma_l^s\|_2^2 \right).$$

Here, μ_l^s and σ_l^s are source statistics. The target statistics μ_l^t and σ_l^t are computed on the current unlabeled target batch. This loss does not use image labels.

3 Problem Formulation

The source domain is the clean CIFAR-10 distribution. It contains image-label pairs (x_s, y_s) sampled from D_s. The target domain contains corrupted CIFAR-10 images sampled from D_t. The labels still correspond to the same ten CIFAR-10 classes, but they are not used during adaptation.

The baseline prediction is obtained by applying the trained backbone and classifier directly to the corrupted batch:

$$\hat{y}_{\text{base}} = \arg\max f_\theta(x_t).$$

The adapted prediction first creates a temporary adapted model. A few gradient steps are applied on the unlabeled target batch using the activation matching loss. The prediction is then computed with the adapted parameters:

$$\hat{y}_{\text{ttt}} = \arg\max f_{\theta'}(x_t), \qquad \theta' = \theta - \alpha \nabla_\theta \mathcal{L}_{\text{ActMAD}}(x_t).$$

In the current implementation, not all parameters are adapted. Most of the model is frozen, and only normalization layer parameters are updated.

The evaluation compares top-1 accuracy before and after adaptation. The improvement is reported in percentage points:

$$\Delta = \text{Acc}_{\text{TTT}} - \text{Acc}_{\text{base}}.$$

A positive value means the adaptation improved accuracy. A negative value means the adaptation damaged accuracy. This sign is important because test-time adaptation should not be described as successful if it improves one setting and fails on another.

4 Methodology

4.1 SimCLR pretraining on CIFAR-10

The first stage trains a self-supervised encoder on CIFAR-10. The loader returns two augmented views of each image. The augmentations used in the repository include random resized cropping, horizontal flipping, color jitter, random grayscale conversion, conversion to tensor, and CIFAR-10 normalization. These transformations are defined in `src/datasets.py` through the two-view transformation used for SimCLR.

The model is defined in `src/self_supervised.py`. It uses a torchvision ResNet18 with the final classification layer replaced by an identity mapping. The output of the backbone is then passed

through a projection head. The projection head is a small multilayer perceptron with a linear layer, ReLU activation, and another linear layer. The output is normalized before the contrastive loss.

For each batch, the training loop computes two projected representations, one for each view. The NT-Xent loss is then applied. The optimizer used in the current code is Adam. A cosine annealing scheduler is also created. The current configuration runs for 200 epochs and prints the training loss at each epoch.

4.2 Feature extraction and projection head

The backbone and projection head serve different roles. The projection head is useful for contrastive pretraining, but the classifier is trained on the raw backbone features. This is standard in SimCLR-style evaluation: the projection space helps the contrastive loss, while the representation before projection is used for downstream tasks.

In the code, the SimCLR forward path returns projected features for the SSL loss. The encoding path returns the raw backbone features. Linear evaluation uses the encoded backbone features, not the projection output. This distinction avoids forcing the final classifier to operate in the contrastive projection space.

4.3 Linear classifier on frozen features

After pretraining, the backbone parameters are frozen. The linear evaluation step trains a linear layer with ten output classes. In the inspected implementation, this classifier is trained with Adam. The classifier is saved as:

```
results/self_supervised/classifier.pt
```

The backbone checkpoint used for TTT is saved as:

```
results/self_supervised/simclr_backbone.pt
```

Linear evaluation is not the final project goal, but it gives an important diagnostic. The final value is 84.23%. This means the self-supervised representation contains useful class information. It is still below strong fully supervised CIFAR-10 baselines, so the later TTT result should be read with that limitation in mind.

4.4 Source activation statistics

Before test-time adaptation, the project computes source activation statistics. The code registers forward hooks on BatchNorm and LayerNorm modules. For each selected layer, it records activations on clean source validation data and computes mean and standard deviation across the appropriate dimensions.

The result is saved as:

```
results/self_supervised/source_stats.pt
```

This file is required by the TTT stage. It represents the clean source-domain activation reference. During adaptation, the corrupted target batch is pushed toward these reference statistics.

4.5 ActMAD-style adaptation at test time

The TTT stage is implemented in `src/test_time_training.py`. The adapter stores the base model and source statistics. For each target batch, the adapter creates a deep copy of the base model. This is important because test-time updates should not permanently modify the original model across unrelated batches. Without a fresh copy, adaptation errors could accumulate from batch to batch.

The copied model is put in training mode. All parameters are frozen first. Then, only parameters inside normalization modules are made trainable. The adaptation optimizer is Adam. The public TTT configuration uses ActMAD, ten adaptation steps per batch, learning rate 10^{-4}, and the source statistics stored in `results/self_supervised/source_stats.pt`.

At each adaptation step, the model processes the unlabeled target batch, the activation matching loss is computed, and the normalization parameters are updated. After the adaptation steps, the copied model is used for prediction. This design is cautious. It does not fine-tune the full network at test time. Updating all weights on a small unlabeled batch would be risky and expensive. Updating only normalization parameters is more limited, but it is less likely to destroy the learned representation.

5 Implementation Details

5.1 Implementation and Repository Structure

To support our experiments, we organized our codebase in a modular way to make it easier to run tasks and ensure reproducibility. The project specifically focuses on Test-Time Training (TTT) and Self-Supervised Learning (SSL).

Rather than placing everything in a single large script, we divided the project into several standard directories. The main folders include `src/` for our core code, `configs/` for experiment settings, `scripts/` for evaluation routines, and `data/` for the datasets. We also included folders like `papers/`, `docs/`, `notebooks/`, `presentation/`, and `report/` to keep all project-related materials neatly in one place.

Everything is executed through a single entry point: `run.py`. From this file, we can launch our two main tasks simply by specifying them in the command line:

- `self_supervised` (or simply `ssl`) for the pre-training phase;
- `test_time_training` (or simply `ttt`) for the adaptation phase.

The core of our work relies on the following key implementation files:

- `run.py`: Handles command-line argument parsing, selects which task to run, loads model checkpoints, and manages the overall execution flow.
- `src/self_supervised.py`: Contains the entire SSL pipeline. This includes the SimCLR architecture, the projection head, the NT-Xent loss, the training loop, and the linear evaluation step. It is also responsible for extracting the source-domain statistics.

- `src/test_time_training.py`: Implements the test-time adaptation logic. This covers the activation hooks needed to gather statistics, the ActMAD loss function, and the TTT adapter itself.
- `src/datasets.py`: Manages our data loading. It includes the loaders for standard CIFAR-10, the stochastic two-view augmentations needed for SimCLR, and the loaders for the corrupted CIFAR-10-C images.
- `src/metrics.py`: Computes the top-1 accuracy so we can reliably compare the baseline model against our TTT approach.
- `scripts/eval_corruption.py`: A helper script we use to systematically evaluate the model's performance across all the different corruption types in CIFAR-10-C.

5.2 Configuration files

The self-supervised configuration is `configs/self_supervised.yaml`. It uses CIFAR-10 at `data/raw`, enables download, and sets the training method to SimCLR. The configuration requests 200 epochs.

Table 1: Self-supervised configuration used for the final SSL run.

Parameter	Value
Task	`self_supervised`
Configuration file	`configs/self_supervised.yaml`
Device in final log	`mps`
Dataset	CIFAR-10
Dataset root	`data/raw`
Backbone	ResNet18
Method	SimCLR
Epochs	200
Batch size	256
Learning rate	0.03
Backbone output	`results/self_supervised/simclr_backbone.pt`
Classifier output	`results/self_supervised/classifier.pt`
Source statistics output	`results/self_supervised/source_stats.pt`

The TTT configuration is `configs/test_time_training.yaml`. It loads the saved SimCLR backbone, classifier, and source statistics. It uses `shot_noise` at severity 5, batch size 16, ten adaptation steps per batch, and learning rate 10^{-4}.

Table 2: Test-time adaptation configuration reported for `configs/test_time_training.yaml`.

Parameter	Value
Task	`test_time_training`
Alias	`ttt`
Configuration file	`configs/test_time_training.yaml`
Dataset	CIFAR-10-C style evaluation
Dataset root in config	`data/raw/cifar10c`
Resolved CIFAR-10-C directory	`data/raw/cifar10c/CIFAR-10-C/`
Corruption in config	`shot_noise`
Severity	5
Batch size	16
Adaptation method	ActMAD
Steps per batch	10
Adaptation learning rate	0.0001
Output directory	`results/test_time_training`

5.3 Checkpoints and result files

The terminal output confirms the following files after self-supervised training:

```
results/self_supervised/classifier.pt
results/self_supervised/simclr_backbone.pt
results/self_supervised/source_stats.pt
```

These files are produced by the self-supervised stage. The TTT stage loads them.

The local CSV output path is:

```
results/test_time_training/results.csv
```

5.4 CIFAR-10-C data path

The TTT configuration sets the dataset root to:

```
data/raw/cifar10c
```

The loader then resolves the official CIFAR-10-C files under:

```
data/raw/cifar10c/CIFAR-10-C/
```

For the corruption `shot_noise`, the expected file is:

```
data/raw/cifar10c/CIFAR-10-C/shot_noise.npy
```

The labels are expected in:

```
data/raw/cifar10c/CIFAR-10-C/labels.npy
```

Each official corruption file contains 50,000 images, with 10,000 images per severity level. The loader selects the slice corresponding to the requested severity. For severity 5, it selects the last 10,000 images.

The final successful TTT command used the same `configs/test_time_training.yaml` path.

6 Experimental Protocol

6.1 Source-domain pretraining

The source-domain pretraining command was:

```
python run.py --task self_supervised --config configs/self_supervised.yaml
```

The alias is:

```
python run.py --task ssl --config configs/self_supervised.yaml
```

The loss log was printed once per epoch. It starts at 5.8654 and reaches 4.5078 at epoch 200. The 200-epoch run completed successfully.

6.2 Linear evaluation

After SimCLR pretraining, `run.py` calls the linear evaluation step. The backbone is frozen and a linear classifier is trained on top of the learned features. The classifier is then evaluated on CIFAR-10. The recorded linear evaluation accuracy was 84.23%.

This result shows that the representation learned class-relevant features. It also leaves room for improvement. A stronger supervised CIFAR-10 model would normally be expected to achieve substantially higher accuracy, so the robustness result should be interpreted together with the quality of the encoder.

6.3 Test-time adaptation setup

The TTT command was:

```
python run.py --task test_time_training --config configs/test_time_training.yaml
```

The alias is:

```
python run.py --task ttt --config configs/test_time_training.yaml
```

This command writes the selected TTT result to `results/test_time_training/results.csv`. The CSV stores the corruption, severity, baseline accuracy, ActMAD accuracy, improvement, and the number of correct predictions.

6.4 All-corruption evaluation

The repository includes an all-corruption script:

```
python scripts/eval_corruption.py
```

The script lists the 15 CIFAR-10-C corruptions:

```
gaussian_noise, shot_noise, impulse_noise, defocus_blur,
glass_blur, motion_blur, zoom_blur, snow, frost, fog,
brightness, contrast, elastic_transform, pixelate,
jpeg_compression
```

This script evaluates the listed corruptions at severity 5 and writes:

```
results/test_time_training/corruption_eval.csv
```

The repository also includes a step sweep:

```
python scripts/sweep_ttt_steps.py
```

It writes:

```
results/test_time_training/ttt_steps_sweep.csv
```

7 Experimental Results

7.1 Final numerical summary

The main numbers are summarized in Table 3. The SSL values come from the recorded training log. The all-corruption TTT values come from the saved corruption-evaluation CSV. The selected `shot_noise` result in `results.csv` is kept as a single-setting artifact, while the all-corruption table is the main robustness result.

Table 3: Final numerical results used in the report.

Experiment	Value
Current self-supervised config epochs	200
Recorded loss log epochs	200
Initial SimCLR loss	5.8654
Final SimCLR loss	4.5078
Linear evaluation accuracy	84.23%
All-corruption evaluation	15 corruptions, severity 5
Mean baseline accuracy	54.82%
Mean ActMAD TTT accuracy	70.22%
Mean improvement	+15.40 percentage points
Corruptions improved	14 / 15

7.2 All-corruption CIFAR-10-C evaluation

Table 4: CIFAR-10-C severity-5 evaluation from `corruption_eval.csv`.

Corruption	Baseline	ActMAD TTT	Improvement
`gaussian_noise`	27.36%	68.02%	+40.66 pp
`shot_noise`	33.02%	69.09%	+36.07 pp
`impulse_noise`	17.89%	59.48%	+41.59 pp
`defocus_blur`	73.76%	74.39%	+0.63 pp
`glass_blur`	35.73%	60.48%	+24.75 pp
`motion_blur`	59.70%	69.23%	+9.53 pp
`zoom_blur`	76.56%	77.91%	+1.35 pp
`snow`	62.32%	70.59%	+8.27 pp
`frost`	49.66%	71.48%	+21.82 pp
`fog`	56.04%	63.33%	+7.29 pp
`brightness`	82.21%	80.66%	-1.55 pp
`contrast`	75.42%	78.43%	+3.01 pp
`elastic_transform`	63.88%	69.38%	+5.50 pp
`pixelate`	38.71%	67.32%	+28.61 pp
`jpeg_compression`	70.05%	73.54%	+3.49 pp
Average	54.82%	70.22%	+15.40 pp

The table shows that the effect of TTT is not just a small average change caused by one outlier. The baseline model is very weak on several severe corruptions, especially `impulse_noise`, `gaussian_noise`, `shot_noise`, `glass_blur`, and `pixelate`. After ActMAD adaptation, these cases move much closer to the accuracy range of the less damaged corruptions. The largest gains appear on the strongest noise-like corruptions: `impulse_noise` (+41.59 pp), `gaussian_noise` (+40.66 pp), and `shot_noise` (+36.07 pp). A strong gain also appears for `pixelate` (+28.61 pp), which suggests that activation matching is useful not only for random noise but also for digital degradations that modify local image structure.

The smaller gains are also informative. For `defocus_blur`, `zoom_blur`, `contrast`, and `jpeg_compression`, the baseline accuracy is already relatively high, so there is less room for improvement. In these cases ActMAD still improves the result, but the gains are more limited. The only negative result is `brightness`, where TTT decreases accuracy by 1.55 percentage points. This is why the result should be described as strong but not universally positive.

7.3 Adaptation-step sweep

Table 5: Step sweep on `shot_noise`, severity 5, from `ttt_steps_sweep.csv`.

Steps per batch	Baseline	ActMAD TTT	Improvement
3	33.02%	53.60%	+20.58 pp
5	33.02%	60.34%	+27.32 pp
10	33.02%	69.09%	+36.07 pp

The sweep shows a clear trend in this setting: more adaptation steps improve the result. This does not mean that increasing the number of steps will always help, but it supports the choice of ten steps for the final evaluation.

7.4 Training loss evolution

The 200-epoch curve in Figure 1 summarizes the loss trajectory from 5.8654 to 4.5078; the largest drop occurs early, and the curve flattens after roughly epoch 70.

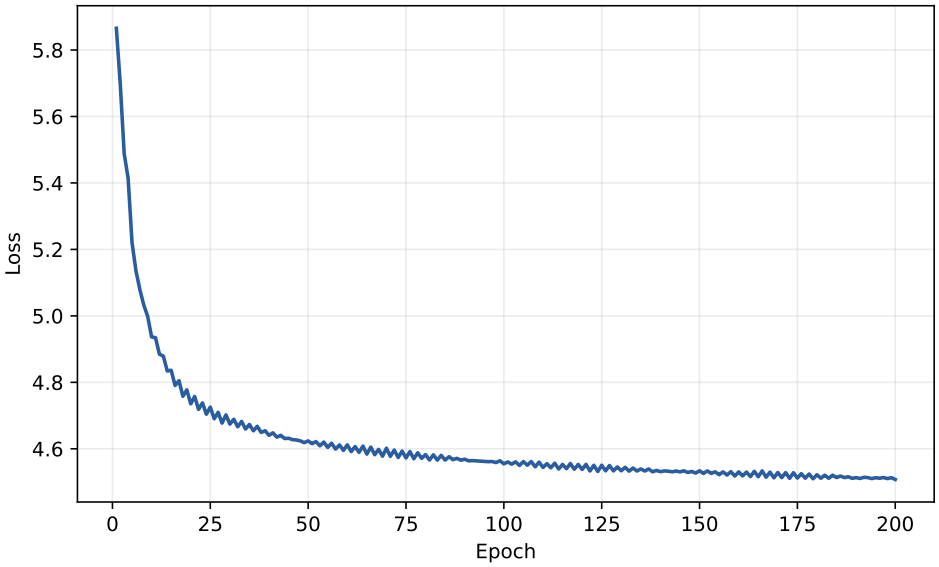

Figure 1: SimCLR training loss over the recorded 200-epoch run.

Table 6: Selected SimCLR loss values from the available training log.

Epoch	1	10	25	50	100	150	200
Loss	5.8654	4.9367	4.7255	4.6238	4.5548	4.5340	4.5078

8 Analysis and Discussion

The evaluation covers all 15 CIFAR-10-C corruption types at severity 5. On average, ActMAD TTT improves accuracy by 15.40 percentage points. It improves 14 corruptions and hurts one. This supports the main idea of the project: matching target activations to source activation statistics can make a corrupted test batch easier for the classifier to handle.

The detailed corruption results show a clear pattern. Noise corruptions are the most favorable case for this implementation. The baseline accuracy is very low for `impulse_noise`, `gaussian_noise`, and `shot_noise`, which means the original representation is strongly disturbed by these shifts. ActMAD gives large gains on all three. This is consistent with the method's objective: noise changes activation means and variances, and the adaptation step explicitly tries to move those statistics back toward the clean source distribution.

Blur and weather corruptions give a more mixed but still positive picture. `glass_blur`, `motion_blur`, `snow`, `frost`, and `fog` all improve, but the gain depends on how much the baseline is damaged. For example, `glass_blur` starts from a low baseline and gains 24.75 percentage points, while `defocus_blur` and `zoom_blur` start from higher baselines and gain only 0.63 and 1.35 points. This suggests that ActMAD is most useful when the corruption creates a large mismatch in internal statistics. When the baseline already handles the corruption well, there is less benefit to adaptation.

The digital and intensity-based corruptions show the main limitation. `pixelate` improves strongly, but `jpeg_compression` and `contrast` improve only slightly, and `brightness` becomes worse. Brightness is especially important because it shows that source-statistic matching is not always aligned with classification accuracy. A brightness change may preserve class-discriminative structure while changing global intensity statistics. Forcing the activations back toward the source statistics can then remove useful information or shift the classifier in the wrong direction.

The step sweep gives a second useful observation. On `shot_noise`, increasing the number of adaptation steps from 3 to 10 improves TTT accuracy from 53.60% to 69.09%. In this experiment, the ActMAD objective benefits from a longer adaptation budget. This should still be treated as setting-specific because more steps can also increase runtime and may be harmful on other corruptions.

Test-time adaptation is not automatically safe. The model adapts without labels. A lower activation matching loss does not always mean better classification. If the target batch is small, unbalanced, or too corrupted, the statistics can be misleading. The optimizer may make the activations look more source-like while losing useful class information. This is why future experiments should keep negative results visible and not only report successful cases.

The SSL stage produces a usable backbone, the linear classifier checks the learned features, source statistics are saved, and TTT loads the saved files and performs adaptation. The results are therefore positive, but the interpretation should stay limited to the reported severity-5 setting.

9 Limitations

The main limitation is still evaluation coverage. The final all-corruption evaluation uses severity 5 only. It does not compare all severity levels from 1 to 5, and it does not report repeated runs with different random seeds.

The settings were only partly compared. The step sweep tests 3, 5, and 10 adaptation steps on `shot_noise`, but the report does not sweep learning rates, batch sizes, or the set of adapted parameters across all corruptions.

ActMAD aligns activation statistics, not labels. A lower activation matching loss does not always mean better classification. This should be checked when reading both positive and negative results.

10 Reproducibility

The main commands from the repository root are:

```
python run.py --task self_supervised --config configs/self_supervised.yaml
python run.py --task test_time_training --config configs/test_time_training.yaml
python run.py --task ssl --config configs/self_supervised.yaml
python run.py --task ttt --config configs/test_time_training.yaml
python scripts/eval_corruption.py
python scripts/sweep_ttt_steps.py
```

The expected order is to run self-supervised training first. This creates the backbone, classifier, and source activation statistics under `results/self_supervised/`. The TTT stage then loads these files. The confirmed saved files are:

```
results/self_supervised/simclr_backbone.pt
results/self_supervised/classifier.pt
results/self_supervised/source_stats.pt
```

The selected TTT output path is:

```
results/test_time_training/results.csv
```

The all-corruption and step-sweep outputs are:

```
results/test_time_training/corruption_eval.csv
results/test_time_training/ttt_steps_sweep.csv
```

For full CIFAR-10-C evaluation, the official `.npy` files and `labels.npy` must exist under:

```
data/raw/cifar10c/CIFAR-10-C/
```

The reported average robustness numbers can be checked directly from the results directory.

The terminal warning observed during the provided run is not central to the scientific method. The MPS warning says that `pin_memory=True` is not supported on MPS, so pinned memory is not used. This can be cleaned by setting `pin_memory=False` when the device is MPS.

11 Final Remarks and Future Work

The project implements a complete pipeline for self-supervised test-time adaptation on image classification. SimCLR trains a ResNet18 backbone on CIFAR-10. A linear classifier evaluates the learned representation. Source activation statistics are computed on clean data. At test time, ActMAD-style adaptation updates normalization parameters on unlabeled corrupted batches before classification.

The recorded result is positive across most severity-5 corruptions: ActMAD TTT improves 14 of the 15 evaluated corruptions and gives an average gain of 15.40 percentage points. The main conclusion is therefore not that the method is universally robust, but that activation matching is a useful test-time signal for many common corruptions, especially noise-like corruptions.

The next practical step is to extend the evaluation to all severity levels and to repeat the sweep for more corruptions. It would also be useful to test several learning rates, several batch sizes, and at least one other test-time adaptation method.

12 References

References

[1] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[2] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[3] Y. Liu, P. Kothari, B. G. van Delft, B. Bellot-Gurlet, T. Mordan, and A. Alahi. TTT++: When Does Self-Supervised Test-Time Training Fail or Thrive? In *Advances in Neural Information Processing Systems*, 2021.

[4] D. Hendrycks and T. Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. In *International Conference on Learning Representations*, 2019.

[5] M. J. Mirza, P. Jané Soneira, W. Lin, M. Kozinski, H. Possegger, and H. Bischof. ActMAD: Activation Matching to Align Distributions for Test-Time-Training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[6] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009.

[7] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.